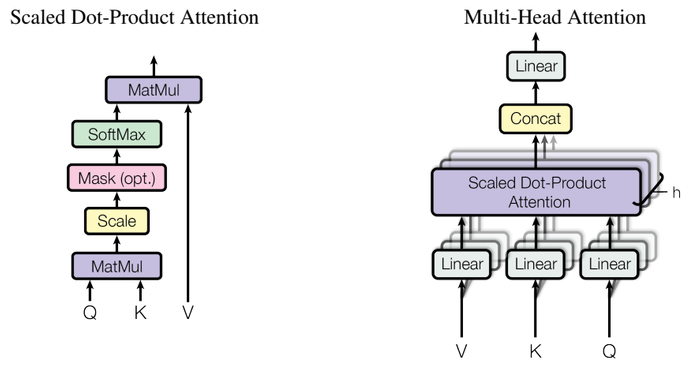


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients [4]. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model}-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional

[4]To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k.

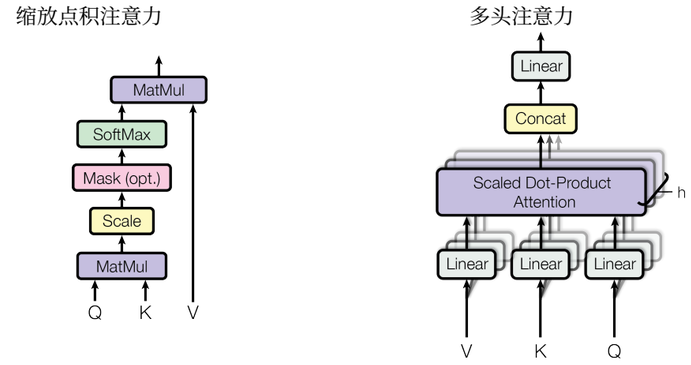


图2：（左）缩放点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

值的加权和，其中分配给每个值的权重由查询与对应键的相容性函数计算得到。

3.2.1 缩放点积注意力

我们将所采用的注意力机制称为“缩放点积注意力”（图2）。输入包括维度为 d_k 的查询和键，以及维度为 d_v 的值。我们计算查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$，再应用 softmax 函数，得到各个值的权重。

在实际计算中，我们将一组查询组成矩阵 Q，同时计算它们的注意力函数。键和值也分别组成矩阵 K 和 V。输出矩阵的计算方式为：

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

最常用的两种注意力函数是加性注意力 [2]，和点积（乘性）注意力。除缩放因子 $\frac{1}{\sqrt{d_k}}$ 外，点积注意力与我们的算法相同。加性注意力使用具有单个隐藏层的前馈网络来计算相容性函数。尽管两者的理论复杂度相近，但在实际计算中，点积注意力速度更快、空间效率更高，因为它可以通过高度优化的矩阵乘法代码实现。

当 d_k 较小时，两种机制的表现相近；但当 d_k [3] 较大时，加性注意力优于未经缩放的点积注意力。我们推测，当 d_k 较大时，点积的量级也会增大，使 softmax 函数进入梯度极小的区域 [4]。为抵消这一影响，我们将点积乘以 $\frac{1}{\sqrt{d_k}}$ 进行缩放。

3.2.2 多头注意力

我们发现，与其对 d_{model} 维的键、值和查询执行一次注意力函数，不如使用不同的、通过学习得到的线性投影，将查询、键和值投影 h 次，分别投影到 d_k、d_k 和 d_v 维。随后，我们对每组投影后的查询、键和值并行执行注意力函数，得到 d_v 维的

[4]为说明点积为何会变大，假设 q 和 k 的各分量都是均值为0、方差为1的独立随机变量。那么，它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为0、方差为 d_k。